

# VIRTUAL SOC LAB

## Security Monitoring & Detection with Wazuh

Cybersecurity Detection & Monitoring Project



### Tools

Wazuh SIEM • Oracle VirtualBox • Windows 11 • Ubuntu Server • Kali Linux

### Lab Network

192.168.56.0/24 | Internal Network: SOC-LAB

### Ethical Use Statement

All testing in this project was performed inside an isolated, owner-controlled virtual lab for defensive cybersecurity learning and monitoring. No external systems were targeted.

### Final Project Report

October 2026

## Executive Summary

---

This project demonstrates the design, implementation, validation, and documentation of a virtual Security Operations Center (SOC) lab using Wazuh SIEM. The environment contains a monitored Windows 11 endpoint, an Ubuntu-based Wazuh server, and a Kali Linux testing machine. The lab was used to generate controlled security events and confirm that the SIEM could collect, correlate, and present useful alerts for analyst review.

Four monitoring outcomes were validated: Windows failed logon detection, file integrity monitoring (FIM), network port-scan detection, and suspicious PowerShell execution detection. Custom Wazuh rules were used to elevate the port-scan and PowerShell scenarios to high-severity alerts, and the PowerShell scenario was mapped to MITRE ATT&CK technique T1059.001 (PowerShell).

### Project Outcome

The final lab successfully monitored the Windows endpoint at 192.168.56.20, detected controlled security activity, preserved alert evidence, and produced a repeatable SOC workflow suitable for a portfolio or entry-level cybersecurity project.

## Report Structure

---

1. Project Overview and Objectives
2. Lab Architecture and Network Design
3. Wazuh Agent Validation
4. Detection Scenario 1 — Failed Logon (Event ID 4625)
5. Detection Scenario 2 — File Integrity Monitoring
6. Detection Scenario 3 — Port Scan Detection
7. Detection Scenario 4 — Suspicious PowerShell Execution
8. Results and Analyst Findings
9. Skills Demonstrated and Lessons Learned
10. Conclusion and Portfolio Summary
11. Appendix A — Key Commands and Queries

# 1. Project Overview and Objectives

---

The goal of the lab was to simulate a small SOC environment in which security telemetry could be generated, collected, analyzed, and validated without exposing any public systems. Oracle VirtualBox provided the virtualization layer, Wazuh provided centralized monitoring and alerting, Windows 11 acted as the monitored endpoint, and Kali Linux generated controlled test traffic.

## Objectives

- Build an isolated three-VM cybersecurity lab with a stable internal network.
- Install and validate a Wazuh agent on Windows 11 and confirm active communication with the Wazuh manager.
- Detect authentication failures using native Windows Security logs.
- Monitor a test path with Wazuh File Integrity Monitoring (FIM).
- Detect controlled network scanning from Kali Linux against the Windows endpoint.
- Detect suspicious PowerShell process creation using Windows Event ID 4688 and a custom high-severity Wazuh rule.
- Document evidence in a form that can be presented in a portfolio, interview, or academic submission.

## Technology Stack

| Component         | Role                          | Validated Detail                               |
|-------------------|-------------------------------|------------------------------------------------|
| Wazuh             | SIEM / centralized monitoring | Dashboard + manager operational; agent v4.14.7 |
| Windows 11 Home   | Monitored endpoint            | Agent ID 002; IP 192.168.56.20                 |
| Ubuntu Server     | Wazuh host                    | Wazuh server IP 192.168.56.10                  |
| Kali Linux        | Controlled security testing   | Testing IP 192.168.56.30                       |
| Oracle VirtualBox | Virtualization platform       | Internal network SOC-LAB + NAT where required  |

## 2. Lab Architecture and Network Design

The SOC lab uses a private 192.168.56.0/24 network. The monitored Windows endpoint communicates with the Wazuh server over the isolated SOC-LAB network, while Kali Linux is used only to generate authorized test activity. A second NAT adapter is retained where Internet access is needed for updates or package installation.

| Kali Linux         | Windows-SOC        | Wazuh Server   |
|--------------------|--------------------|----------------|
| 192.168.56.30      | 192.168.56.20      | 192.168.56.10  |
| Controlled testing | Monitored endpoint | SIEM / Manager |

Authorized scans →

Windows telemetry →

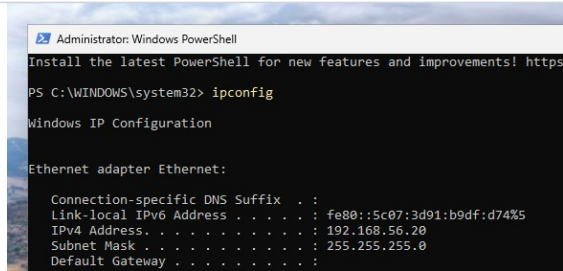
**Internal Network: SOC-LAB | Subnet: 192.168.56.0/24**

The screenshot displays the VirtualBox configuration window for a VM named 'Day3-Final-Test'. The 'Details' tab is selected, showing various hardware settings. The 'System' tab is highlighted in blue. The 'Network' section shows two adapters: 'Adapter 1' is 'Intel PRO/1000 MT Desktop' connected to 'Internal Network, 'SOC-LAB'', and 'Adapter 2' is 'Intel PRO/1000 MT Desktop' connected to 'NAT'. Other settings include 4096 MB Base Memory, 2 Processors, and SATA ports for 'Day3-Final.vdi' and 'VBoxGuestAdditions.iso'.

| Category       | Setting                                   | Value                                                   |
|----------------|-------------------------------------------|---------------------------------------------------------|
| General        | Name                                      | Day3-Final-Test                                         |
|                | Operating System                          | Windows 11 (64-bit)                                     |
| System         | Base Memory                               | 4096 MB                                                 |
|                | Processors                                | 2                                                       |
|                | Boot Order                                | Floppy, Optical, Hard Disk                              |
|                | TPM Type                                  | 2.0                                                     |
|                | EFI                                       | Enabled                                                 |
|                | Secure Boot                               | Enabled                                                 |
| Acceleration   | Nested Paging, Hyper-V Paravirtualization |                                                         |
| Display        | Video Memory                              | 128 MB                                                  |
|                | Graphics Controller                       | VBoxSVGA                                                |
|                | Remote Desktop Server                     | Disabled                                                |
|                | Recording                                 | Disabled                                                |
| Storage        | Controller                                | SATA                                                    |
|                | SATA Port 0                               | Day3-Final.vdi (Normal, 80.00 GB)                       |
|                | SATA Port 1                               | [Optical Drive] VBoxGuestAdditions.iso (50.64 MB)       |
| Audio          | Host Driver                               | Default                                                 |
|                | Controller                                | Intel HD Audio                                          |
| Network        | Adapter 1                                 | Intel PRO/1000 MT Desktop (Internal Network, 'SOC-LAB') |
|                | Adapter 2                                 | Intel PRO/1000 MT Desktop (NAT)                         |
| USB            | USB Controller                            | xHCI                                                    |
|                | Device Filters                            | 0 (0 active)                                            |
| Shared folders | None                                      |                                                         |
| Description    | None                                      |                                                         |

Figure 1. VirtualBox configuration for the recovered Windows-SOC endpoint, showing the SOC-LAB internal network and NAT adapters.

## Validated IP Addresses



```

Administrator: Windows PowerShell
Install the latest PowerShell for new features and improvements! https://aka.ms/PowerShellLatest

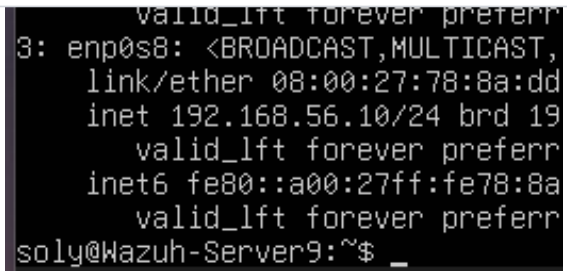
PS C:\WINDOWS\system32> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::5c07:3d01:b9df:d74%5
    IPv4 Address. . . . . : 192.168.56.20
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :
  
```

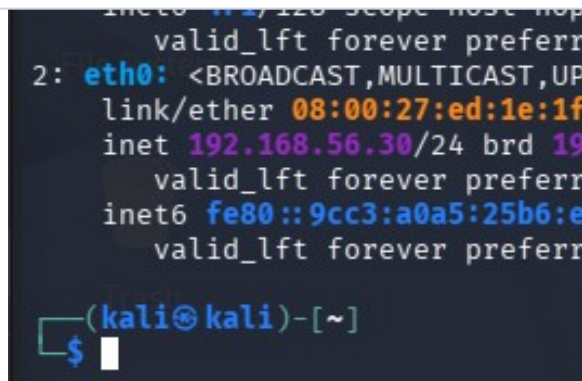
Figure 2A. Windows-SOC — 192.168.56.20



```

valid_lft forever prefer
3: enp0s8: <BROADCAST,MULTICAST,UP>
   link/ether 08:00:27:78:8a:dd
   inet 192.168.56.10/24 brd 192.168.56.255
   valid_lft forever preferred
   inet6 fe80::a00:27ff:fe78:8a:dd
   valid_lft forever preferred
soly@Wazuh-Server9:~$ _
  
```

Figure 2B. Wazuh Server — 192.168.56.10



```

inet6 fe80::2/128 scope host noprefixroute
   valid_lft forever preferred
2: eth0: <BROADCAST,MULTICAST,UP>
   link/ether 08:00:27:ed:1e:1f
   inet 192.168.56.30/24 brd 192.168.56.255
   valid_lft forever preferred
   inet6 fe80::9cc3:a0a5:25b6:e
   valid_lft forever preferred

(kali@kali)-[~]
$
  
```

Figure 2C. Kali Linux — 192.168.56.30

### 3. Wazuh Agent Validation

Before running detection scenarios, the Windows endpoint was validated in the Wazuh Endpoints view. The agent was online, assigned IP 192.168.56.20, and reported Wazuh agent version v4.14.7. This confirms that telemetry from the endpoint could reach the Wazuh manager.

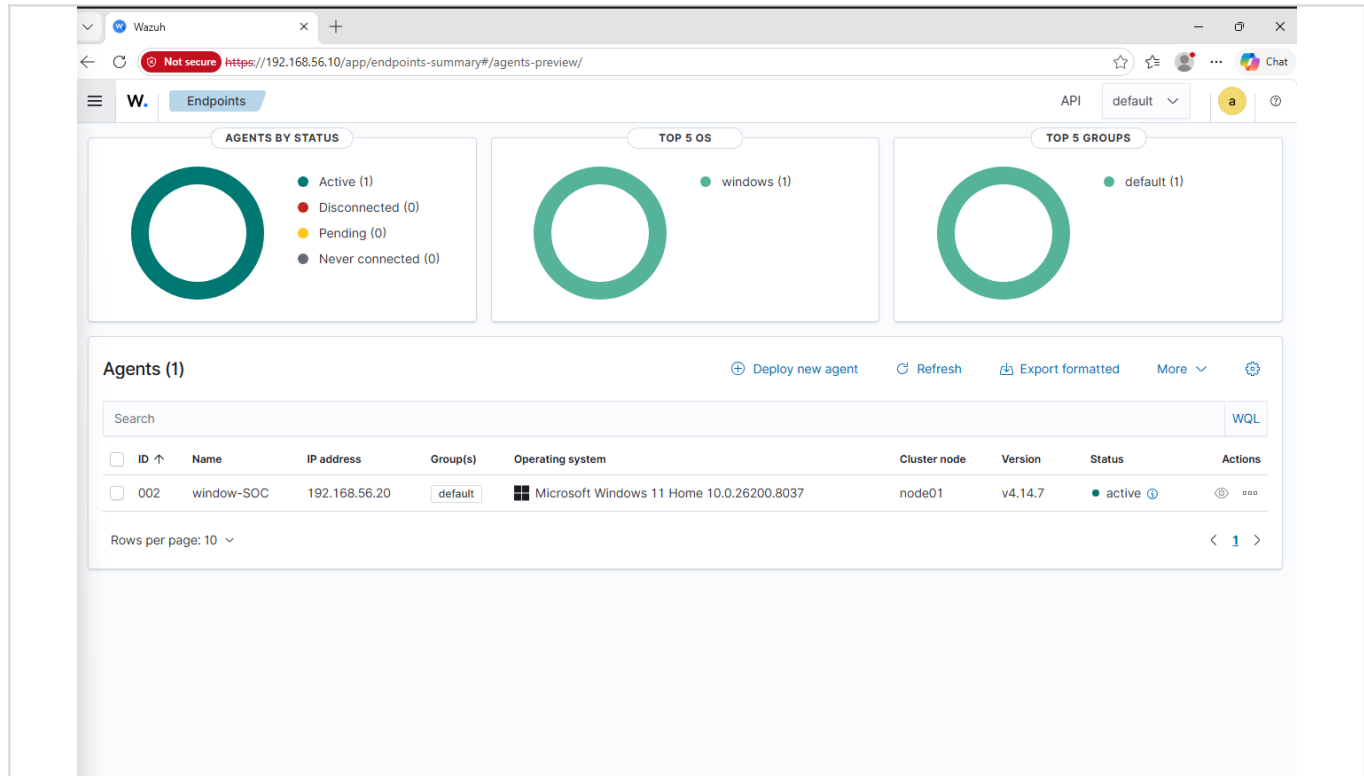


Figure 3. Wazuh Endpoints view showing agent ID 002 (window-SOC) active at 192.168.56.20.

#### Validation Result

Agent status: ACTIVE. This established a trusted baseline before reviewing authentication, FIM, network, and PowerShell alerts.

## 4. Detection Scenario 1 — Failed Logon (Event ID 4625)

### Scenario

A dedicated local test account (soc-test) was used to generate an intentionally failed logon attempt. Windows Security auditing recorded the authentication failure and Wazuh ingested the event for investigation.

### Detection Evidence

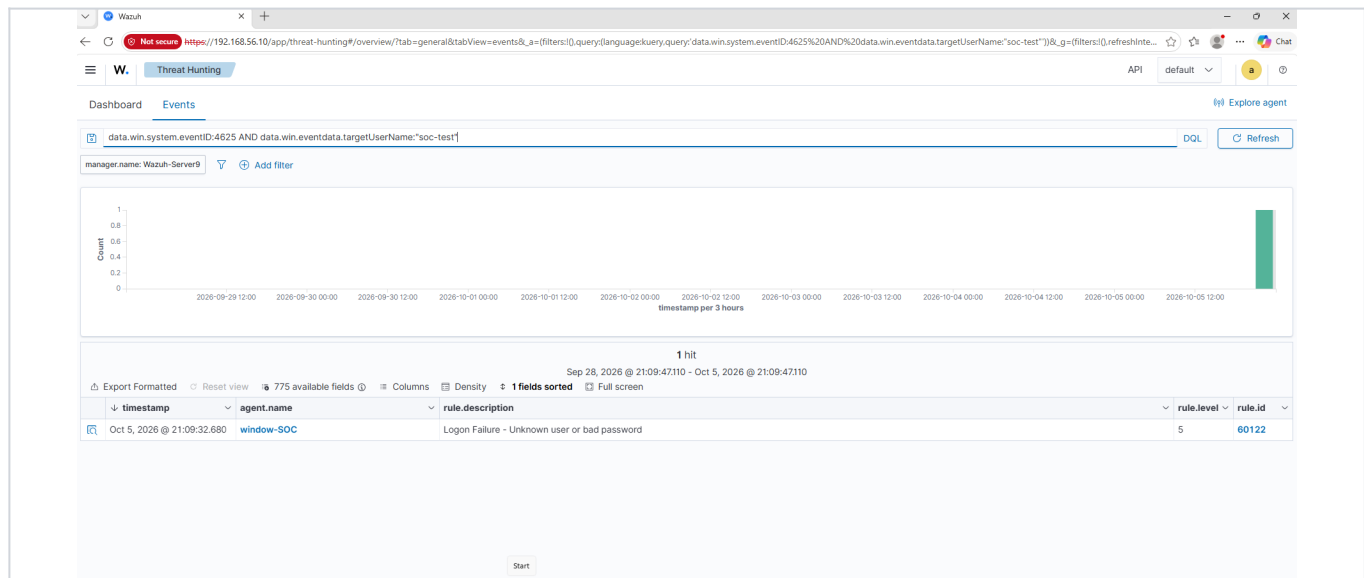


Figure 4. Wazuh Threat Hunting result for Event ID 4625 filtered to target user soc-test; rule 60122, level 5.



Figure 5. Event detail confirming targetUserName = soc-test.

### Analyst Interpretation

- Windows Event ID 4625 indicates that an account failed to log on.
- The Wazuh rule description identified the cause as an unknown user or bad password.
- The event demonstrates that native Windows authentication telemetry is successfully collected and searchable in the SIEM.
- In a production SOC, repeated 4625 events from the same source, account, or time window could support brute-force or credential-attack investigation.

## 5. Detection Scenario 2 — File Integrity Monitoring

### Scenario

Wazuh File Integrity Monitoring was configured to monitor the controlled test path C:\SOC-Test. A file named test.txt was created and then modified so that the SIEM could demonstrate both file-addition and file-change visibility.

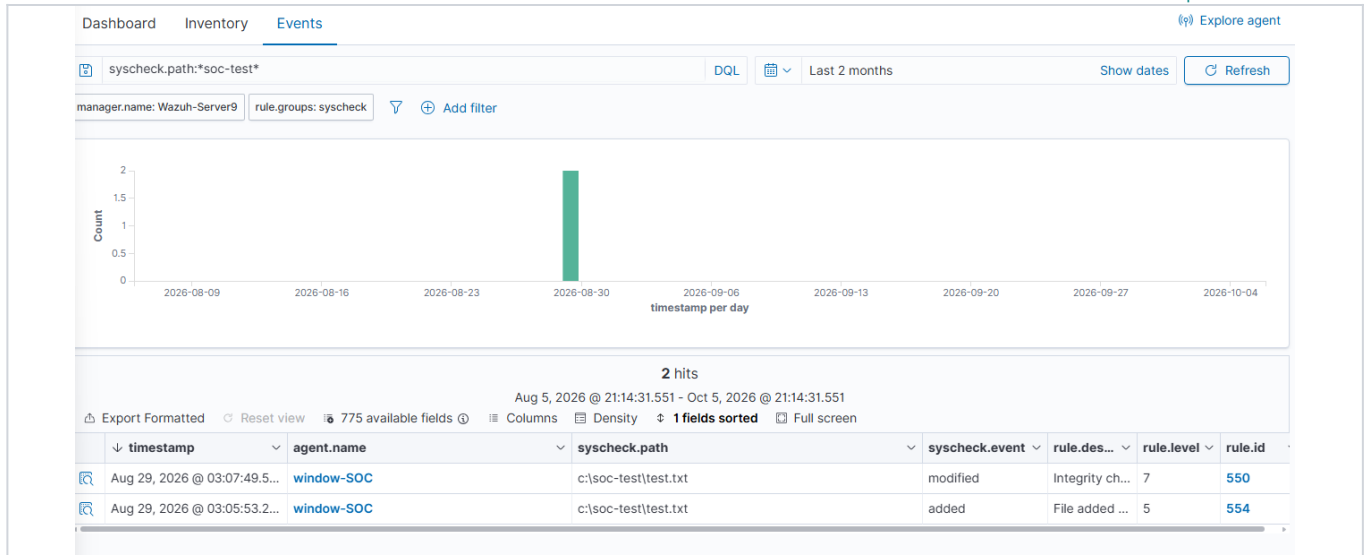


Figure 6. FIM evidence for C:\SOC-Test\test.txt showing both added and modified events.

## Observed Results

| Path                 | Action   | Rule ID | Level |
|----------------------|----------|---------|-------|
| C:\SOC-Test\test.txt | added    | 554     | 5     |
| C:\SOC-Test\test.txt | modified | 550     | 7     |

### SOC Value

FIM provides evidence of changes to monitored files. This is useful for detecting unauthorized modification of configuration files, scripts, binaries, or other sensitive assets.

## 6. Detection Scenario 3 — Port Scan Detection

### Scenario

Kali Linux (192.168.56.30) generated a controlled network scan against the monitored Windows endpoint (192.168.56.20). Windows Firewall telemetry was collected by Wazuh and a custom correlation rule was used to identify repeated scan-like activity.

### Custom Detection

- Base/local rule: 100100 — network scan related event matching.
- Correlation rule: 100101 — level 10 alert after repeated matching activity within a short timeframe.
- Alert description: “SOC LAB: Possible port scan detected from 192.168.56.30 to 192.168.56.20.”

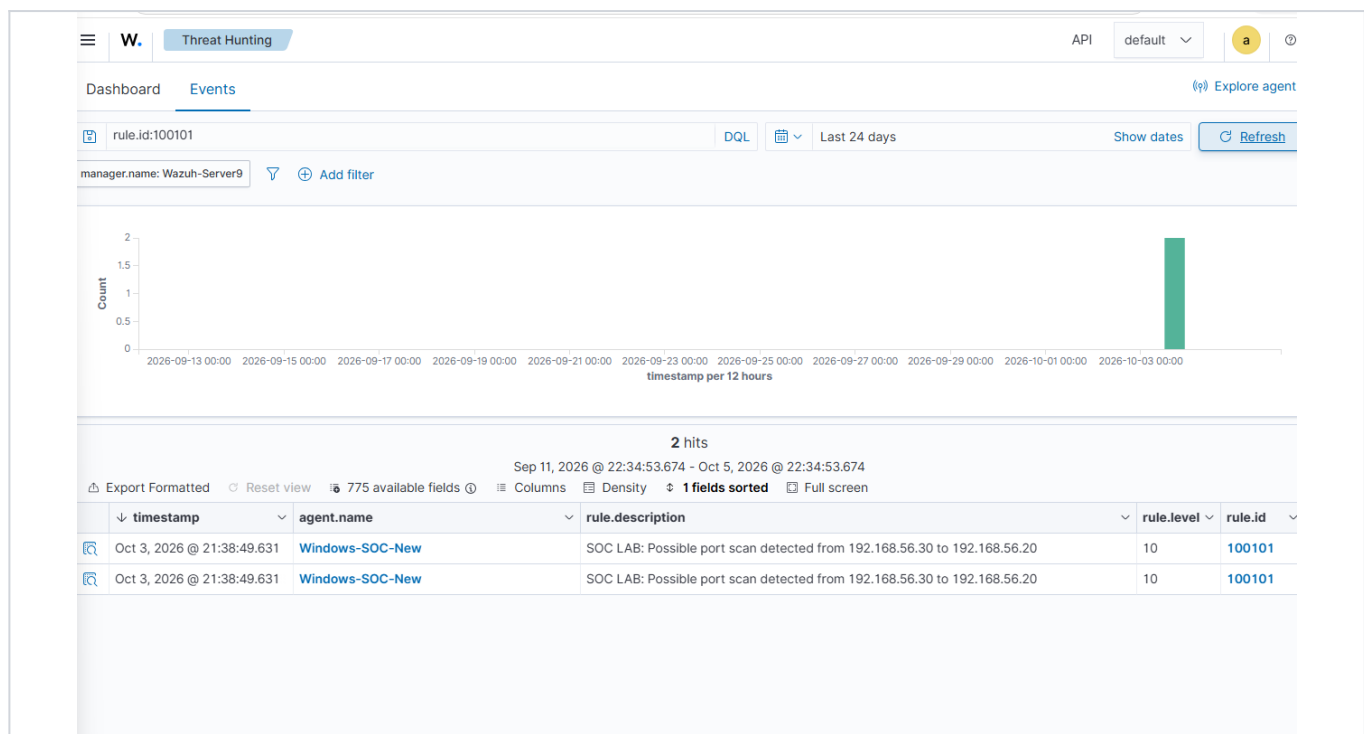


Figure 7. Wazuh custom rule 100101 detecting a possible port scan from Kali (192.168.56.30) to Windows (192.168.56.20).

### Analyst Interpretation

- The alert correlates multiple network events rather than relying on a single packet or port.
- The source and destination addresses match the documented lab architecture, confirming end-to-end visibility.
- A level 10 alert provides a clear prioritization signal for an analyst to inspect the scanning source and target.

## 7. Detection Scenario 4 — Suspicious PowerShell Execution

### Scenario

Windows process-creation auditing was enabled so that Event ID 4688 would include the process command line. A safe PowerShell command was launched with `-NoProfile` and `-ExecutionPolicy Bypass` to emulate a suspicious execution pattern without running a malicious payload.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-DAY5'"
```

### Custom Detection Rule

- Wazuh base process-creation rule: 67027.
- Custom rule: 100102.
- Severity: level 10.
- Detection condition: command line contains `ExecutionPolicy Bypass`.
- MITRE ATT&CK mapping: T1059.001 — PowerShell.

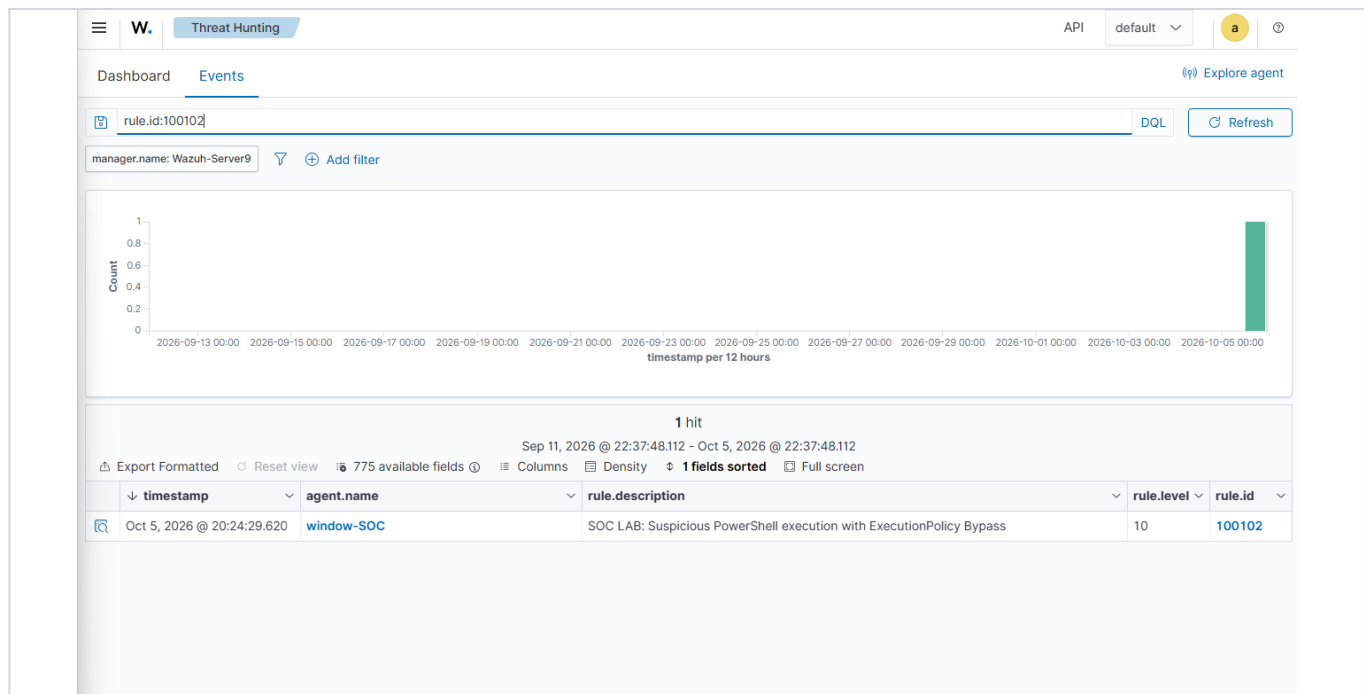
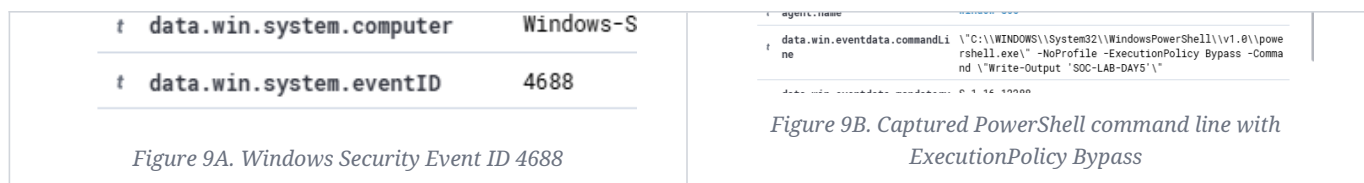


Figure 8. Wazuh alert from custom rule 100102 for suspicious PowerShell execution with ExecutionPolicy Bypass.



### Analyst Interpretation

- Event ID 4688 confirms that a new process was created and provides process-level telemetry.

- Command-line logging exposes execution arguments that would otherwise be invisible in a process-name-only alert.
- ExecutionPolicy Bypass is not automatically malicious, but it is a strong investigation signal when combined with context, parent process, user, and other endpoint activity.
- Mapping the alert to MITRE ATT&CK gives the detection a standard threat-behavior reference useful for SOC reporting and coverage tracking.

## 8. Results and Analyst Findings

| Scenario          | Primary Data           | Rule / Event | Severity | Result   |
|-------------------|------------------------|--------------|----------|----------|
| Failed logon      | Windows Security       | 4625 / 60122 | Level 5  | Detected |
| File added        | Wazuh FIM              | 554          | Level 5  | Detected |
| File modified     | Wazuh FIM              | 550          | Level 7  | Detected |
| Port scan         | Firewall / local rules | 100101       | Level 10 | Detected |
| PowerShell bypass | Process Creation 4688  | 100102       | Level 10 | Detected |

### Key Findings

- The Wazuh agent remained active and reliably delivered Windows endpoint telemetry to the manager.
- Windows authentication failures were searchable at both event and username level.
- FIM successfully differentiated file creation from later modification.
- Custom rule logic converted raw network events into a clear high-severity port-scan alert.
- Process-creation auditing plus command-line collection enabled a high-confidence PowerShell behavior alert with MITRE ATT&CK context.

### Detection Workflow Demonstrated

#### Attack / Activity → Log → Wazuh → Rule → Alert → Analyst Review

Each scenario followed the same SOC pattern: controlled activity generated telemetry, the endpoint or firewall logged it, Wazuh collected and decoded the data, a rule evaluated the event, an alert was raised, and the evidence was reviewed in Threat Hunting or FIM views.

## 9. Skills Demonstrated and Lessons Learned

### Technical Skills Demonstrated

- VirtualBox virtual machine management, snapshot use, storage recovery, and network-adapter configuration.
- Linux administration on Ubuntu and Kali, including SSH, service management, file inspection, and command-line troubleshooting.
- Wazuh manager, dashboard, Windows agent, Threat Hunting, File Integrity Monitoring, and local rule configuration.

- Windows Security auditing, Event ID 4625 analysis, Event ID 4688 process creation, and command-line audit configuration.
- Network scanning validation in an isolated lab and correlation of firewall events into a high-severity alert.
- Custom SIEM rule design, severity tuning, evidence collection, and MITRE ATT&CK mapping.
- Structured evidence gathering for incident analysis and project documentation.

## Troubleshooting and Resilience

During the project, the Windows virtual machine required recovery from a saved virtual disk/snapshot chain. After recovery, the endpoint was revalidated by confirming the C:\SOC-Test path, the Wazuh service, network connectivity to 192.168.56.10, and active agent status in the dashboard. This recovery process reinforced the importance of snapshots, verification steps, and change control in a lab environment.

## Lessons Learned

- A detection is only complete when the full data path is validated, not just when a command executes.
- Time filters and stale/duplicate agent entries can hide or confuse evidence; filters should be verified before concluding that data is missing.
- Custom rules should be tested with known-safe activity, then validated in both raw logs and the dashboard.
- Snapshots and clear naming reduce recovery time and make experimentation safer.

## 10. Conclusion and Portfolio Summary

---

The Virtual SOC Lab successfully demonstrated a practical defensive monitoring workflow across endpoint, file, network, and process telemetry. The project moved beyond installation by proving that the environment could detect controlled security events, generate meaningful alerts, and support analyst investigation using Wazuh.

The strongest outcomes were the custom level-10 port-scan detection and the custom PowerShell detection mapped to MITRE ATT&CK T1059.001. Together with authentication and FIM evidence, these scenarios provide a compact but realistic portfolio example of SOC fundamentals: log collection, detection engineering, triage, correlation, evidence validation, and documentation.

### Portfolio One-Line Summary

Built a three-VM Virtual SOC lab using Wazuh, Windows 11, Ubuntu, and Kali Linux; implemented and validated detections for failed logons, file integrity changes, port scanning, and suspicious PowerShell execution, including custom level-10 Wazuh rules and MITRE ATT&CK mapping.

## Suggested CV Entry

Virtual SOC Lab — Wazuh SIEM | Built an isolated Windows/Kali/Ubuntu SOC environment in VirtualBox; deployed a Wazuh agent, analyzed Windows Security and FIM events, and created

custom level-10 detections for port scanning and suspicious PowerShell execution (MITRE ATT&CK T1059.001).

### Suggested Interview Talking Point

“I built a virtual SOC lab from scratch and focused on validating the whole detection pipeline. For example, I enabled Windows process-creation auditing, captured Event ID 4688 with command-line arguments, wrote a Wazuh rule to detect ExecutionPolicy Bypass, mapped it to MITRE T1059.001, then confirmed the alert in Threat Hunting. I also built a correlated port-scan alert and documented the evidence.”

## Appendix A — Key Commands and Queries

---

The commands below were used only inside the isolated SOC-LAB environment.

### Windows Validation

```
Test-Path "C:\SOC-Test"
Get-Service WazuhSvc
ping 192.168.56.10
```

### Process Creation Auditing

```
auditpol /set /subcategory:"Process Creation" /success:enable
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v
ProcessCreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
```

### Safe PowerShell Test Activity

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "Write-Output 'SOC-LAB-DAY5'"
```

### Useful Wazuh Queries

```
data.win.system.eventID:4625 AND data.win.eventdata.targetUserName:"soc-test"
syscheck.path:*soc-test*
rule.id:100101
rule.id:100102
```

### Wazuh Manager Validation

```
sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

### End of Report

All evidence shown in this report was captured from the author's isolated virtual SOC environment.